

THEORY -

1. Explain the role of NumPy in the Python scientific computing ecosystem.**

NumPy (Numerical Python) is a core library used for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices along with a collection of mathematical functions to operate on these arrays.

Role of NumPy:

- Provides efficient array operations
- Forms the base for libraries like Pandas, SciPy, and Scikit-learn
- Used in data analysis, machine learning, and numerical computation

It is the foundation of most data science and scientific applications in Python.

2. Why is NumPy considered faster than pure Python computations?**

NumPy is faster than pure Python because:

- It uses **optimized C code** internally
- Supports **vectorized operations** (no need for loops)
- Uses **less memory** due to homogeneous data types

Operations on arrays are executed in compiled code, making them much faster than Python loops.

3. What types of mathematical operations can be performed using NumPy?**

NumPy supports a wide range of mathematical operations such as:

- Arithmetic operations: addition, subtraction, multiplication, division
- Statistical operations: mean, median, standard deviation
- Linear algebra: matrix multiplication, transpose
- Trigonometric functions: sin, cos, tan
- Logical operations and comparisons

Example:

```
import numpy as np
a = np.array([1,2,3])
print(a + 2)
```

4. Explain indexing and slicing in NumPy arrays.**

Indexing:

- Accessing elements using their position
- Starts from index 0

Slicing:

- Extracting a subset of array elements
- Syntax: `array[start:end:step]`

Example:

```
import numpy as np
a = np.array([10,20,30,40])
print(a[1])      # Indexing → 20
print(a[1:3])    # Slicing → [20, 30]
```

Works similarly to Python lists but faster and more powerful.

5. How does NumPy integrate with libraries like:**

a. Pandas

- Pandas uses NumPy arrays internally
- DataFrame and Series are built on NumPy

b. Matplotlib

- NumPy arrays are used to plot graphs
- Helps in visualizing numerical data

c. Scikit-learn

- Machine learning models use NumPy arrays as input
- Efficient for numerical computations

NumPy acts as a backbone connecting all these libraries.

```
import numpy as np
import time

# 1. Create a NumPy array from a list
print("Q1: Create array from list")
arr1 = np.array([1, 2, 3, 4])
print(arr1)
print()

# 2. Perform basic arithmetic operations
print("Q2: Arithmetic operations")
a = np.array([1,2,3])
```

```

b = np.array([4,5,6])
print("Addition:", a + b)
print("Multiplication:", a * b)
print()

# 3. Find sum and mean
print("Q3: Sum and Mean")
arr3 = np.array([10, 20, 30])
print("Sum:", np.sum(arr3))
print("Mean:", np.mean(arr3))
print()

# 4. Reshape a 1D array
print("Q4: Reshape array")
arr4 = np.array([1,2,3,4,5,6])
print(arr4.reshape(2,3))
print()

# 5. Access elements using indexing
print("Q5: Indexing")
arr5 = np.array([10,20,30,40])
print(arr5[2])
print()

# 6. Matrix addition and multiplication
print("Q6: Matrix operations")
m1 = np.array([[1,2],[3,4]])
m2 = np.array([[5,6],[7,8]])
print("Addition:\n", m1 + m2)
print("Multiplication:\n", np.dot(m1, m2))
print()

# 7. Generate random numbers
print("Q7: Random numbers")
print(np.random.rand(3))
print()

# 8. Standard deviation and variance
print("Q8: Std and Variance")
arr8 = np.array([1,2,3,4,5])
print("Std:", np.std(arr8))
print("Variance:", np.var(arr8))
print()

# 9. Mathematical functions
print("Q9: Math functions")
arr9 = np.array([0, np.pi/2, np.pi])
print("sin:", np.sin(arr9))
print("cos:", np.cos(arr9))
print()

```

```

# 10. Compare list vs NumPy array
print("Q10: List vs NumPy")
lst = [1,2,3]
arr10 = np.array(lst)
print("List *2:", lst*2)          # repeats list
print("Array *2:", arr10*2)       # multiplies values
print()

# 11. Vectorized operations
print("Q11: Vectorization performance")
lst = list(range(1000000))
arr = np.array(lst)

start = time.time()
lst_res = [x*2 for x in lst]
print("List time:", time.time() - start)

start = time.time()
arr_res = arr * 2
print("NumPy time:", time.time() - start)
print()

# 12. Solve linear equations
print("Q12: Linear equations")
A = np.array([[2,3],[1,2]])
B = np.array([8,5])
solution = np.linalg.solve(A,B)
print(solution)
print()

# 13. Broadcasting
print("Q13: Broadcasting")
arr13 = np.array([1,2,3])
print(arr13 + 5)
print()

# 14. Analyze large dataset
print("Q14: Large dataset analysis")
data = np.random.randint(1,100, size=1000)
print("Mean:", np.mean(data))
print("Max:", np.max(data))
print("Min:", np.min(data))
print()

# 15. Optimize computations
print("Q15: Optimized computation")
arr15 = np.arange(1,1000000)
print("Sum using NumPy:", np.sum(arr15))

```

Q1: Create array from list
[1 2 3 4]

Q2: Arithmetic operations
Addition: [5 7 9]
Multiplication: [4 10 18]

Q3: Sum and Mean
Sum: 60
Mean: 20.0

Q4: Reshape array
[[1 2 3]
[4 5 6]]

Q5: Indexing
30

Q6: Matrix operations
Addition:
[[6 8]
[10 12]]
Multiplication:
[[19 22]
[43 50]]

Q7: Random numbers
[0.36866048 0.61412154 0.38004613]

Q8: Std and Variance
Std: 1.4142135623730951
Variance: 2.0

Q9: Math functions
sin: [0.0000000e+00 1.0000000e+00 1.2246468e-16]
cos: [1.0000000e+00 6.123234e-17 -1.0000000e+00]

Q10: List vs NumPy
List *2: [1, 2, 3, 1, 2, 3]
Array *2: [2 4 6]

Q11: Vectorization performance
List time: 0.054491281509399414
NumPy time: 0.005436420440673828

Q12: Linear equations
[1. 2.]

Q13: Broadcasting
[6 7 8]

Q14: Large dataset analysis
Mean: 51.432
Max: 99
Min: 1

```
Q15: Optimized computation  
Sum using NumPy: 499999500000
```

Conclusion –

In this experiment, we learned how to perform numerical computations using NumPy. We practiced array creation, arithmetic operations, reshaping, and indexing. We also explored advanced topics like vectorization, broadcasting, and solving linear equations. NumPy improves performance and efficiency, making it essential for scientific computing and data analysis.